DG_Room Components and Population

1. `id`

- Set in `generate_room(int id, GeneratorState *g)` as `r.id = id`.
- Source: `data_gen/src/datagen.c` in `generate_room`.

2. `width`, `height`

- Set in `init_room_dimensions(&r, g)` using base config `room_width/room_height` plus random variance.
- Enforced minimum of 4.
- Source: `data_gen/src/datagen.c` in `init_room_dimensions`.

3. `floor_grid`

- Allocated and initialized in `alloc_and_init_floor_grid(&r)` to a `width*height` boolean array.
- Borders set to `false` (walls), interior `true` (floor).
- Then modified by multiple steps: `generate_sokoban_layout`, `generate_switches`, `align_pushable_for_switch`, `ensure_switch_reachability`, `enforce_entity_floor_invariant`, `enforce_portal_clearance`, `enforce_portal_reachability`.
- Assigned to `r.floor_grid` at end of `generate_room`.
- Source: `data_gen/src/datagen.c` in `alloc_and_init_floor_grid` and `generate_room`.

4. `portal_count`

- Computed in `compute_entity_counts(&r, g)` as `portals_per_room +/- portals_variance`, clamped to at least 1.
- Source: `data_gen/src/datagen.c` in `compute_entity_counts`.

5. `portals`

- Allocated in `generate_portals(&r, g)` as an array of `DG_Portal`.
- Each portal:
  - `id = i`
  - `type = DG_PORTAL_DOOR`
  - `neighbor_id = -1` (until inter-room linking)
  - `required_switch_id = -1` (unless switch is created)
  - `(x, y)` chosen either on walls or interior based on `portal_placement_mode`.
- Inter-room connectivity updates `neighbor_id` in `connect_rooms`.
- A portal may get `required_switch_id = 0` in `generate_switches` when a switch is created.
- Sources: `data_gen/src/datagen.c` in `generate_portals`, `connect_rooms`, `generate_switches`.

6. `treasure_count`

- Computed in `compute_entity_counts(&r, g)` as `treasures_per_room * scale +/- variance`, clamped to >= 0.
- Scale depends on `difficulty` (lower difficulty yields more treasures).
- Source: `data_gen/src/datagen.c` in `compute_entity_counts`.

7. `treasures`

- Allocated in `generate_entities(&r, g)` as an array of `DG_Treasure`.
- Each treasure:
  - `global_id = g->next_treasure_id++`
  - `type` randomized in `[0, treasure_types-1]`
  - `value = 1`
  - `name = strdup("Treasure")`
  - `(x, y)` random interior tile within `entity_padding`.
- Note: `g->next_treasure_id` is initialized by `calloc` (0) and only set to `1000` *after* all rooms are generated in `start_datagen`, so IDs start at 0 in current flow.
- Sources: `data_gen/src/datagen.c` in `generate_entities` and `start_datagen`.

8. `pushable_count`

- Computed in `compute_entity_counts(&r, g)` as `pushables_per_room * (difficulty/5.5) +/- variance`, clamped to >= 0.
- Source: `data_gen/src/datagen.c` in `compute_entity_counts`.

9. `pushables`

- Allocated in `generate_entities(&r, g)` as an array of `DG_Pushable`.
- Each pushable:
  - `id = i`
  - `type` randomized in `[0, pushable_types-1]`
  - `flags = 0`
  - `name = strdup("Pushable")`
  - `(x, y)` random interior tile within `entity_padding`.
- If a switch is created, `align_pushable_for_switch` may reposition the first pushable (`pushables[0]`) adjacent to the switch.
- Sources: `data_gen/src/datagen.c` in `generate_entities` and `align_pushable_for_switch`.

10. `switch_count`

- Set in `generate_switches(&r, g, floor)`.
- Either `0` (if no portals or no pushables), or `1` otherwise.
- Source: `data_gen/src/datagen.c` in `generate_switches`.

11. `switches`

- Allocated in `generate_switches` when switch_count becomes 1.
- The single switch:

    - `id = 0`
    - `portal_id = portal_idx` (a random portal)
    - `x, y` placed adjacent to that portal if free, otherwise random interior.
- Also sets the selected portal's `required_switch_id = 0`.
- Sources: `data_gen/src/datagen.c` in `generate_switches`.

Lifecycle Summary (Order of Population)

In `generate_room`:

1. `id`
2. `width`, `height`
3. `floor_grid` allocated and base-initialized
4. `portal_count`, `treasure_count`, `pushable_count`
5. `portals` array (positions, defaults)
6. `floor_grid` modified by `generate_sokoban_layout`
7. `treasures` and `pushables` arrays
8. `switches` (optional) and linking to a portal
9. Reposition pushable for switch (optional)
10. Ensure reachability, enforce invariants on `floor_grid`
11. Assign `r.floor_grid = floor`

Then, after all rooms are generated, `connect_rooms` assigns `neighbor_id` for each portal to build a connected graph.

INI File options

Below is a detailed summary of **all `.ini` options recognized by the generator**, based on the parser in `data_gen/src/datagen.c` and the reference config `assets/treasure_runner.ini`.

How the parser works

- Lines starting with `#` or blank lines are ignored.
- Keys are parsed as `key=value` under a `[section]`.
- Unknown sections/keys are ignored.
- Defaults are set in code before parsing.

[metadata]

- `name`
    - Present in `assets/treasure_runner.ini` for humans, but ignored by datagen
- `seed`
    - `random` makes generation use `time(NULL)`.
    - Otherwise parsed as an integer seed.

[world]

- `num_rooms`
  - Total rooms generated.
- `difficulty`
  - Scales treasure and pushable counts:
    - Treasures scale down as difficulty increases.
    - Pushables scale up as difficulty increases.

[room]

- `width`
  - Base room width before variance.
- `height`
  - Base room height before variance.
- `width_variance`
  - Random offset in `[-width_variance, +width_variance]` for each room.
- `height_variance`
  - Random offset in `[-height_variance, +height_variance]` for each room.
- `entity_padding`
  - Minimum distance from walls when placing entities/portals in interior mode.
  - Used by portals (interior), treasures, and pushables.

[portals]

- `portals_per_room`
  - Base portal count per room.
- `portals_variance`
  - Random offset in `[-portals_variance, +portals_variance]`.
  - Clamped so each room has at least 1 portal.
- `portal_placement`
  - `interior` places portals inside the room.
  - Any other value (including `walls`) places portals on room boundaries.
- `connectivity_chance`
  - Probability (0–100) that an unconnected portal becomes an extra inter-room link.

[maze]

- `interior_wall_density`
  - Controls density of interior walls in the maze generation.
  - Higher values generally mean more open space.

[treasures]

- `treasures_per_room`
  - Base treasures per room (scaled by difficulty).
- `treasures_variance`
  - Random offset in `[-treasures_variance, +treasures_variance]`.

- `treasure_types`
  - Number of distinct types; each treasure gets a random type in `[0, treasure_types-1]`.

[pushables]

- `pushables_per_room`
  - Base pushables per room (scaled by difficulty).
- `pushables_variance`
  - Random offset in `[-pushables_variance, +pushables_variance]`.
- `pushable_types`
  - Number of distinct types; each pushable gets a random type in `[0, pushable_types-1]`.

[charset]

- `wall`, `floor`, `player`, `pushable`, `treasure`, `portal`, `switch_off`, `switch_on`
  - Single-character tokens used by the UI / renderer.
  - Supports escapes: `\s` (space), `\t`, `\n`, `\\`.
  - If a value is missing or empty, defaults apply.

Minimum set defaults (if not specified in the ini file) These are hard-coded in `parse_ini_file`:

- `seed=12345`, `num_rooms=8`, `difficulty=3`
- `width=12`, `height=10`, `width_variance=2`, `height_variance=2`, `entity_padding=1`
- `portals_per_room=2`, `portals_variance=1`, `portal_placement=walls`
- `connectivity_chance=30`, `interior_wall_density=75`
- `treasures_per_room=3`, `treasures_variance=1`, `treasure_types=1`
- `pushables_per_room=1`, `pushables_variance=0`, `pushable_types=1`
- charset defaults: `# . @ O $ * = +`

Right now there is no .ini option to enable/disable switches. The generator always attempts to create a single switch per room if the room ends up with at least 1 portal and 1 pushable. Easy to change that and push a new version of the .so to their repos.

From a gameplay perspective, switches are pressure plates that temporarily unlock a specific portal.

A switch controls exactly one portal. When a switch exists, that portal's required_switch_id is set, meaning the portal is locked unless the switch is pressed. Pressing a switch is done by moving a pushable onto the switch tile.

The generator only creates a switch if the room has at least 1 portal and 1 pushable. At most one switch per room.

The generator tries to place the switch adjacent to the controlled portal (so the dependency is visible), and aligns a pushable near it to make the puzzle solvable. The room floor layout is adjusted to ensure the pushable can reach the switch tile.